云原生软件技术 Lab 1：场地预约系统开发与 Docker 容器化

一、实验主题

开发一个简单的场地预约系统，并使用 Docker 和 Docker Compose 完成前端、后端 API 与数据库的容器化部署。

本实验是课程三阶段项目的第一阶段。后续 Lab 将在本系统上增加 Worker、Kubernetes 编排和可观测性能力，因此各小组应保留完整代码仓库和清晰的项目结构。

二、实验目标

完成本实验后，学生应能够：

1. 开发一个包含前端、Web API 和关系数据库的简单应用；
2. 使用 Web API 实现前后端通信；
3. 使用关系数据库持久化场地和预约数据；
4. 理解应用、进程、镜像与容器之间的关系；
5. 为前端和后端分别编写 Dockerfile；
6. 使用 Docker Compose 定义和运行多容器应用；
7. 使用环境变量实现配置外部化；
8. 使用 Volume 保存数据库数据；
9. 验证系统能够被其他人按照文档重复构建和运行。

三、组织方式

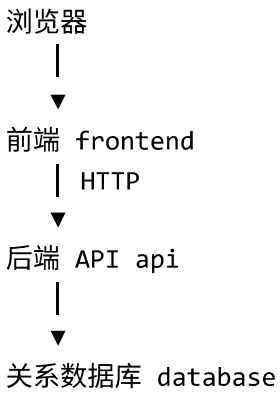
- 以小组为单位完成；
- 建议每个组创建一个 Git 仓库，并同步代码；
- 可以使用生成式 AI 辅助需求分析、编码、编写配置和排查错误，但小组成员必须能够解释、修改和验证最终提交的代码与配置。

第一部分：Lab 1 实践要求

一、系统总体要求

1.1 系统运行结构

系统至少包含三个运行组件：



三个组件应分别承担以下职责：

组件	职责
`frontend`	提供用户界面，调用后端 Web API
`api`	实现业务规则、参数校验和数据访问
`database`	持久化场地与预约数据

1.2 技术栈

- 后端框架和关系数据库可自行选择；
- 前端可以使用主流框架，也可以使用简单的 HTML、CSS 和 JavaScript；
- 接口采用 HTTP Web API；
- 容器工具使用 Docker；
- 多容器应用使用 Docker Compose v2。

二、业务功能要求

2.1 场地实体

场地至少应包含以下信息：

- 场地编号；
- 场地名称；
- 所在位置；
- 可容纳人数；
- 开放时间；
- 场地状态，例如开放、停用。

2.2 预约实体

预约至少应包含以下信息：

- 预约编号；
- 场地编号；
- 预约人或预约组织；
- 预约日期；
- 开始时间；
- 结束时间；
- 使用目的；
- 预约状态，例如有效、已取消。

2.3 必做业务功能

系统至少应支持：

1. 查询场地列表；
2. 查询场地详情；
3. 创建场地；
4. 创建预约；
5. 查询预约列表；
6. 查询预约详情；
7. 取消预约；
8. 在创建预约时检查场地、时间和输入数据的有效性；

9. 拒绝同一场地在同一时间段内的冲突预约。

三、后端 Web API 要求

后端至少提供以下等价能力。路径可以调整，但必须在 README 和实验报告中列出实际接口。例如：

```
GET    /api/venues
GET    /api/venues/{id}
POST   /api/venues
POST   /api/reservations
GET    /api/reservations
GET    /api/reservations/{id}
DELETE /api/reservations/{id}
```

也可以使用显式取消接口代替 `DELETE`：

```
POST /api/reservations/{id}/cancel
```

API 应满足：

1. 使用 JSON 作为主要请求与响应格式；
2. 对必填字段、时间范围和场地状态进行校验；
3. 对不存在的场地或预约返回明确错误；
4. 对预约冲突返回明确错误；
5. 不把异常堆栈直接返回给普通用户；
6. 数据必须写入关系数据库，不能只保存在进程内存中；
7. 数据库连接信息必须从环境变量或外部配置读取。

建议使用语义明确的 HTTP 状态码：

场景	建议状态码
查询或取消成功	`200 OK`
创建成功	`201 Created`
请求参数错误	`400 Bad Request`
资源不存在	`404 Not Found`

场景	建议状态码
预约时间冲突	`409 Conflict`
未处理的服务端错误	`500 Internal Server Error`

四、前端要求

前端至少应提供以下交互：

1. 展示场地列表；
2. 查看场地基本信息；
3. 创建场地预约；
4. 展示预约记录；
5. 取消预约；
6. 显示预约成功信息；
7. 显示时间冲突、参数错误等失败信息。

前端要求：

- 通过 HTTP 调用后端 API；
- 不直接连接数据库；
- 不在前端实现唯一的预约冲突判断；
- 后端地址应可以通过构建参数、环境配置、反向代理或其他合理方式配置；
- 页面能够完成业务操作即可，不要求复杂 UI。

五、数据库要求

1. 可自行选择使用的关系数据库；
2. 至少包含场地表和预约表；
3. 明确主键和场地—预约之间的关联关系；
4. 提供数据库初始化脚本、迁移文件或框架迁移机制；
5. 提供少量可用于演示的场地种子数据；
6. 预约数据必须持久化；
7. 取消预约可以采用逻辑取消或物理删除，但必须在设计和报告中说明；
8. 数据库密码不得写死在源代码或 Docker 镜像中。

六、Dockerfile 要求

6.1 前端 Dockerfile

应做到：

- 可以独立构建前端镜像；
- 前端构建产物能够通过容器提供服务；
- 不把本地依赖缓存和无关文件复制到镜像；
- 合理使用 `.dockerignore`；
- 明确容器监听端口。

6.2 后端 Dockerfile

应做到：

- 可以独立构建后端镜像；
- 最终运行镜像不应包含不必要的源码、编译工具和缓存；
- 使用前台进程作为容器主进程；
- 通过环境变量读取数据库配置；
- 不在镜像中写入数据库密码；
- 合理使用 `.dockerignore`；
- 建议使用非 root 用户运行。

6.3 镜像要求

- 使用明确、合理的基础镜像版本；
- 镜像名称和标签应有含义；
- 不要求推送到公共镜像仓库；
- 不要求提交镜像导出文件；
- 应保证其他人可以根据源码和 Dockerfile 重新构建镜像。

七、Docker Compose 要求

项目根目录或 `deploy/` 目录中应提供 `compose.yaml`，至少定义：

```
frontend
api
database
```

7.1 服务要求

`frontend`

- 使用前端镜像；
- 对外发布访问端口；
- 能够访问 `api`；
- 不直接访问 `database`。

`api`

- 使用后端镜像；
- 能够通过 Compose 服务名连接数据库；
- 从环境变量或外部配置读取数据库信息。

`database`

- 使用标准数据库镜像；
- 通过环境变量配置数据库名、用户名和密码；
- 使用命名 Volume 或绑定目录持久化数据；
- 配置合理的 `healthcheck`。

7.2 网络要求

- 前端、API 和数据库通过 Compose 网络通信；
- API 使用服务名 `database` 或实际定义的数据库服务名连接数据库；
- 只有确需从宿主机访问的端口才对外发布；
- 数据库端口原则上不必对外发布；如开发调试需要发布，应在报告中说明。

7.3 配置要求

普通配置可以放入：

- `compose.yaml` 的 `environment`；
- `.env`；

- `env_file` 指定的文件；
- 应用支持的外部配置文件。

提交时必须提供：

```
.env.example
```

要求：

- `.env.example` 只包含变量名和实验示例值；
- 真实 `.env` 应加入 `.gitignore`；
- 不得提交真实账号、密码、Token 或访问密钥；
- 不得在 Dockerfile 中写入环境相关密码。

八、运行与验收要求

8.1 基本功能验证

至少完成：

1. 打开前端页面；
2. 查询场地列表；
3. 创建正常预约；
4. 查询刚创建的预约；
5. 创建时间冲突的预约，并确认后端拒绝；
6. 取消预约；
7. 在取消后的时间段重新创建预约，并确认结果符合设计。

8.2 容器验证

执行：

```
docker compose up --build -d
docker compose ps
```

至少确认：

- 三个服务容器已创建；

- `frontend` 可以访问；
- `api` 可以连接 `database` ；
- 数据库健康检查正常；
- 服务日志中没有持续重复的严重错误。

8.3 数据持久化验证

建议采用以下流程：

1. 创建一条预约并记录预约编号；
2. 执行：

```
docker compose down
```

3. 再次启动：

```
docker compose up -d
```

4. 查询该预约，确认数据仍然存在；
5. 使用 `docker volume ls` 查看项目 Volume。

实验报告必须记录持久化验证过程和结果。

九、提交形式

每组提交一个压缩包或 Git 仓库地址，其中包括源代码，配置文件等。

另外，每组提交一个Lab1实验报告。

实验报告要求：

- 系统架构、数据库结构与接口说明；
- 每一个自定义服务的docker文件及说明；
- compose文件及说明；
- 系统部署、启动、执行记录：可包含使用的命令，以及命令执行结果；
- 主要的系统使用用例流程与记录；
- 小组分工与执行情况；
- 问题与解决过程（可选）。

十、提交时间

报告提交截止时间：2026/11/2。

提交至elearning